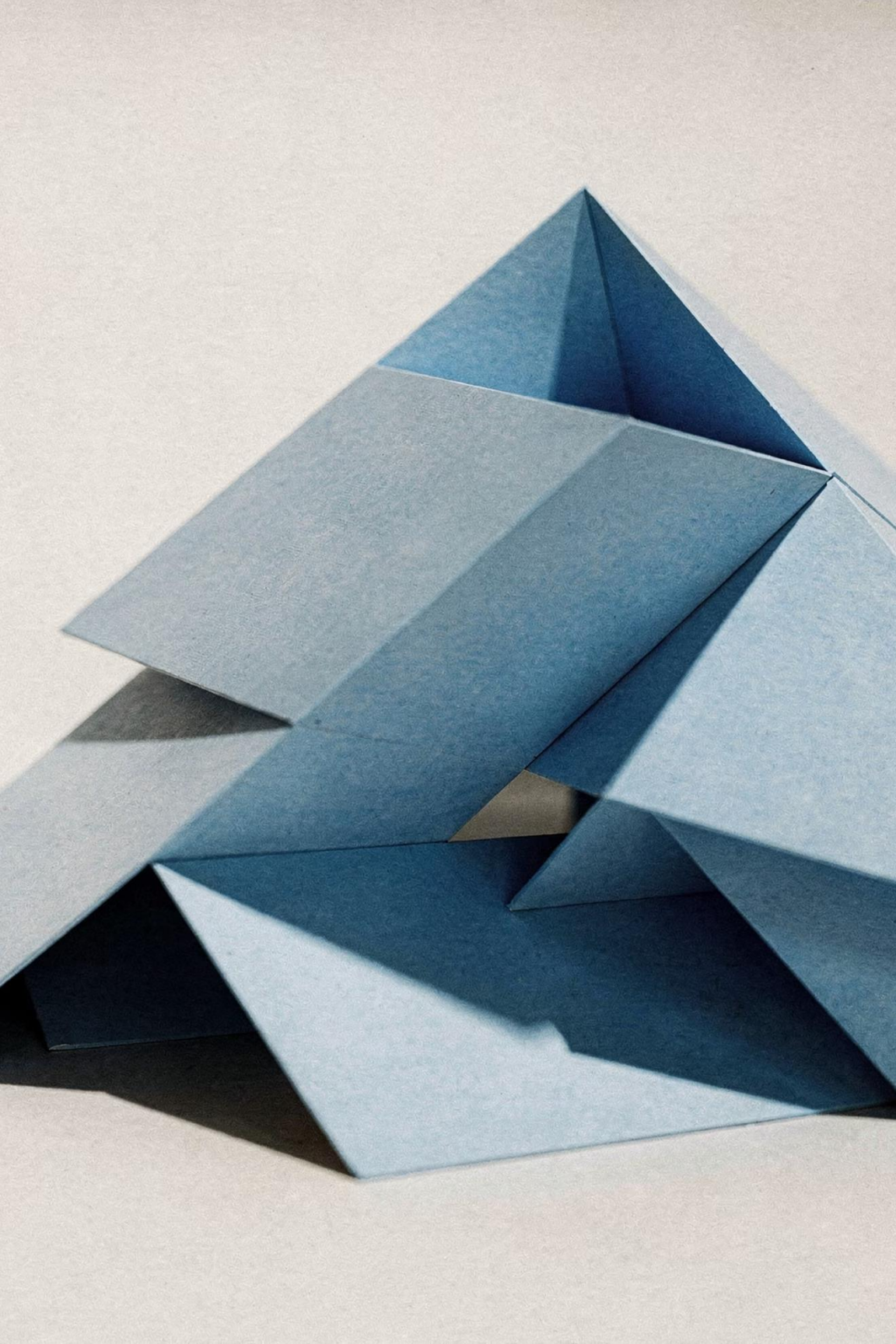


Angular Moderno

31 - Organizacao Profissional

2026

SENAI - CRTI



31 — Organização Profissional

Nesta aula vamos evoluir o projeto para um padrão mais profissional de arquitetura, separando responsabilidades de forma clara. Até aqui, nossa aplicação já funciona completamente. Agora vamos reorganizar o acesso às regras de negócio, preparando o projeto para crescer de forma escalável.

Service vs Facade

Diferença e papel de cada camada

Separação de
Responsabilidades

Organização profissional do código

Refatoração Segura

Sem alterar nenhum comportamento



Nenhuma funcionalidade nova será criada nesta aula. O objetivo é reorganizar o código existente de forma profissional.

Revisão do Projeto Atual

Até a aula anterior, nosso projeto já possui uma base sólida e funcional. Veja o que já foi construído:

Carrinho Centralizado

Gerenciado por service dedicado

Autenticação com JWT

Login simulado com token

Guards de Rota

Proteção de rotas privadas

Interceptor HTTP

Injeção automática de token

Área Administrativa

Controle de acesso por perfil

⚠ Os componentes, guards e interceptors acessam diretamente os services. Isso funciona, mas não é o ideal em projetos maiores. Precisamos criar uma camada intermediária para organizar melhor o acesso à lógica da aplicação.

⚠ OBS: O carrinho com estado centralizado em service/facade, mas ainda com exibição visual parcialmente ligada à listagem de produtos.

Conceito — Service vs Facade

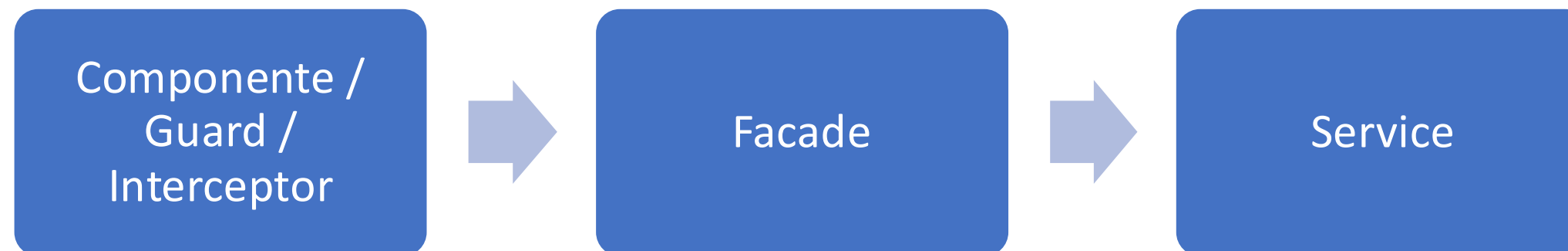
Service

- Contém a regra de negócio
- Manipula dados
- Gerencia estado
- Executa ações reais (login, carrinho, etc)

Facade

- NÃO cria nova regra de negócio
- Atua como uma camada de acesso
- Simplifica o uso dos services
- Evita acoplamento direto com a lógica interna

Fluxo após esta aula



Com a introdução da Facade, os componentes deixam de conhecer os detalhes internos dos services, tornando o código mais limpo e fácil de manter.

Etapa 1 — Criar Pasta de Facades

O primeiro passo é criar a estrutura de pastas que irá abrigar todas as facades da aplicação.

1

Caminho da Pasta

`src/app/core/facades`

2

Por que em **core**?

A pasta **core** centraliza tudo que é global e essencial para o funcionamento da aplicação.

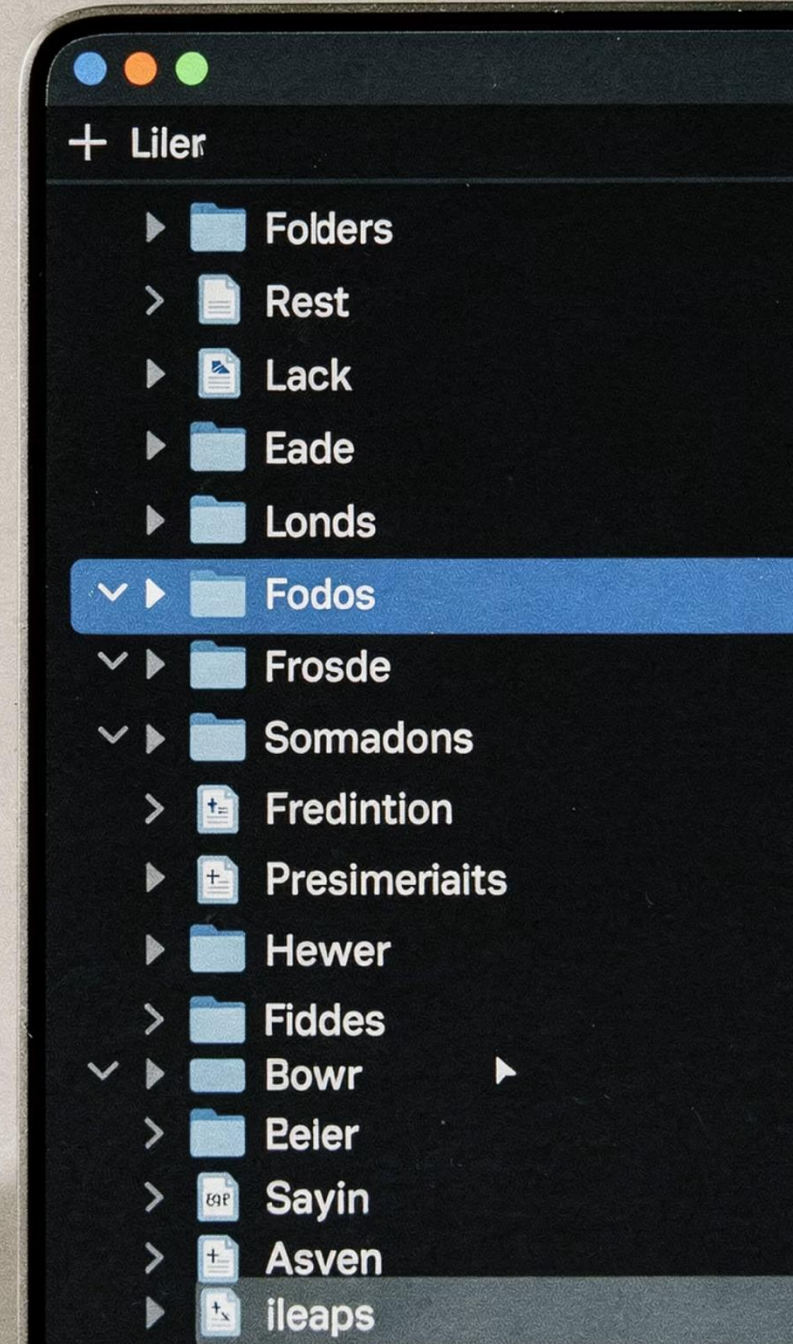
3

Responsabilidade

Centralizar todas as facades da aplicação, controlando o acesso às regras globais do sistema.



Esta pasta faz parte da camada **core**, pois controla acesso às regras globais do sistema.



Etapa 2 — Criar `carrinho.facade.ts`

Arquivo

`src/app/core/facades/carrinho.facade.ts`

Objetivo

Criar uma camada de acesso para o carrinho, evitando que os componentes acessem diretamente o `CarrinhoService`.

O que esta facade expõe

- Itens do carrinho
- Quantidade de produtos
- Total do pedido
- Ação de adicionar produto
- Ação de limpar carrinho

 A `CarrinhoFacade` não implementa nenhuma lógica nova — ela apenas delega as chamadas para o `CarrinhoService`.

Código — carrinho.facade.ts

src/app/core/facades/carrinho.facade.ts

```
import { Injectable, inject } from '@angular/core';

import { CarrinhoService } from '../services/carrinho.service';

type ItemCarrinho = {
  nome: string;
  preco: number;
};

@Injectable({
  providedIn: 'root',
})
export class CarrinhoFacade {
  // A facade passa a ser a camada usada pelos componentes,
  // evitando que as telas dependam diretamente dos detalhes internos do service.
  private carrinhoService = inject(CarrinhoService);

  // Sinais expostos para leitura pelas telas.
  itens = this.carrinhoService.itens;
  quantidade = this.carrinhoService.quantidade;
  total = this.carrinhoService.total;
  carrinhoVazio = this.carrinhoService.carrinhoVazio;

  // Ação de alto nível para adicionar produto ao carrinho.
  adicionarProduto(produto: ItemCarrinho) {
    this.carrinhoService.adicionar(produto);
  }

  // Ação de alto nível para limpar o carrinho.
  limparCarrinho() {
    this.carrinhoService.limpar();
  }
}
```

Etapa 3 — Criar `auth.facade.ts`

Arquivo

`src/app/core/facades/auth.facade.ts`

Objetivo

Centralizar o acesso à autenticação da aplicação, expondo uma interface simples para todos os consumidores.

O que esta facade expõe

- Usuário atual
- Status de login
- Perfil (admin ou usuário)
- Método de login
- Método de logout



A facade **NÃO** implementa a lógica de autenticação. Ela apenas delega para o `AuthService`.

Código — auth.facade.ts

src/app/core/facades/auth.facade.ts

```
import { Injectable, inject } from '@angular/core';

import { AuthService } from '../services/auth.service';

@Injectable({
  providedIn: 'root',
})
export class AuthFacade {
  // A facade centraliza o acesso da aplicação à autenticação.
  // Componentes, guards e header passam a usar esta camada simplificada.
  private authService = inject(AuthService);

  // Sinais de autenticação expostos para leitura.
  usuarioAtual = this.authService.usuarioAtual;
  estaLogado = this.authService.estaLogado;
  ehAdmin = this.authService.ehAdmin;
  token = this.authService.token;
```

```
// Ação de alto nível para login.
realizarLogin(email: string, senha: string): boolean {
  return this.authService.login(email, senha);
}

// Ação de alto nível para logout.
sair() {
  this.authService.logout();
}

// Método mantido para integrações técnicas, como interceptor HTTP.
obterToken(): string | null {
  return this.authService.obterToken();
}

// Método de leitura do perfil atual.
obterPerfil() {
  return this.authService.obterPerfil();
}
}
```

Etapa 4 — Atualizar `header.ts`

Arquivo: `src/app/shared/layout/header/header.ts`

ANTES

- Uso direto de `AuthService`
- Uso direto de `CarrinhoService`

DEPOIS

- Uso de `AuthFacade`
- Uso de `CarrinhoFacade`

- ✓ O header deixa de conhecer a implementação interna dos services e passa a usar uma interface mais simples e desacoplada.



[Home](#) [Products](#) [About](#) [Contact](#)

Código — header.ts

src/app/shared/layout/header/header.ts

```
import { Component, inject } from '@angular/core';
import { Router, RouterLink } from '@angular/router';
import { MatToolbarModule } from '@angular/material/toolbar';
import { MatButtonModule } from '@angular/material/button';

import { CarrinhoFacade } from '../../../core/facades/carrinho.facade';
import { AuthFacade } from '../../../core/facades/auth.facade';

@Component({
  selector: 'app-header',
  imports: [MatToolbarModule, MatButtonModule, RouterLink],
  templateUrl: './header.html',
  styleUrls: ['./header.css'],
})
export class Header {
  // o Header deixa de acessar diretamente os services.
  // Agora ele consome facades, que simplificam o acesso ao carrinho e à autenticação.
  private carrinhoFacade = inject(CarrinhoFacade);
  private authFacade = inject(AuthFacade);
  private router = inject(Router);

  // Sinais recebidos da facade do carrinho.
  quantidade = this.carrinhoFacade.quantidade;

  // Sinais recebidos da facade de autenticação.
  estaLogado = this.authFacade.estaLogado;
  usuarioAtual = this.authFacade.usuarioAtual;

  sair() {
    // Logout feito pela facade, não mais diretamente pelo service.
    this.authFacade.sair();
    this.router.navigateByUrl('/login');
  }
}
```

Etapa 5 – Atualizar `lista-produtos.ts`

Arquivo: `src/app/features/produtos/lista-produtos/lista-produtos.ts`

1

Antes

```
this.carrinhoService.adicionar(produto)
```

2

Depois

```
this.carrinhoFacade.adicionarProduto(produto)
```

Sinais também atualizados

quantidade

Agora via facade

total

Agora via facade

✔ Objetivo: remover dependência direta do `CarrinhoService` no componente de listagem de produtos.

Código — lista-produtos.ts

```
import { Component, signal, computed, effect, inject } from '@angular/core';
import { MatButtonModule } from '@angular/material/button';
import { ProdutosService } from '../core/services/produtos.service';
import { CarrinhoFacade } from '../core/facades/carrinho.facade';
import { Produto } from '../produto/produto';

@Component({
  selector: 'app-lista-produtos',
  imports: [Produto, MatButtonModule],
  templateUrl: './lista-produtos.html',
  styleUrls: ['./lista-produtos.css'],
})
export class ListaProdutos {
  // INJECTS
  produtosService = inject(ProdutosService);
  carrinhoFacade = inject(CarrinhoFacade);

  quantidadeCarrinho = this.carrinhoFacade.quantidade;
  totalCarrinho = this.carrinhoFacade.total;

  // === SIGNALS
  produtos = signal<{ nome: string; preco: number }[]>([]);
  produtoSelecioneado = signal<string | null>(null);
  carregando = signal(true);
  erro = signal<string | null>(null);

  // === COMPUTED
  totalProdutos = computed(() => this.produtos().length);

  valorTotal = computed(() => {
    return this.produtos().reduce((total, item) => total + item.preco, 0);
  });

  // === CONSTRUTOR
  constructor() {
    this.carregarProdutos();
    effect(() => {
      console.log('Lista de produtos alterada:', this.produtos());
    });
    effect(() => {
      console.log('Valor total atualizado:', this.valorTotal());
    });
    effect(() => {
      if (typeof document !== 'undefined') {
        document.title = `${this.totalProdutos()} Minha Loja`;
      }
    });
  }
}
```

src/app/features/produtos/lista-produtos/lista-produtos.ts

```
// === MÉTODO HTTP (API)
carregarProdutos() {
  this.erro.set(null);
  this.carregando.set(true);

  this.produtosService.buscarProdutos().subscribe({
    next: (dados) => {
      const produtos = this.produtosService.transformarProdutos(dados);
      this.produtos.set(produtos);
      this.carregando.set(false);
    },
    error: (erro) => {
      console.error('Erro ao carregar produtos:', erro);
      this.erro.set('Erro ao carregar produtos. Verifique sua conexão e tente novamente.');
```

```
      this.carregando.set(false);
    },
  });
}

// === DEMAIS MÉTODOS
exibirProduto(nome: string) {
  this.produtoSelecioneado.set(nome);
}

adicionarProduto() {
  this.produtos.update((listaAtual) => [...listaAtual, { nome: 'Teclado', preco: 250 }]);
}

substituirProdutos() {
  this.produtos.set([{ nome: 'Produto novo', preco: 999 }]);
}

adicionarAoCarrinho(produto: { nome: string; preco: number }) {
  this.carrinhoFacade.adicionarProduto(produto);
}
}
```

Etapa 6 – Atualizar `lista-produtos.html`

Arquivo: `src/app/features/produtos/lista-produtos/lista-produtos.html`

ANTES

Referências a `carrinhoService` no template

DEPOIS

Referências a `carrinhoFacade` no template

Pontos de substituição no template

itens

Lista de produtos no carrinho

quantidade

Contador de itens

total

Valor total do pedido



Objetivo: manter consistência com a nova arquitetura em toda a camada de interface.

Código — lista-produtos.html

src/app/features/produtos/lista-produtos/lista-produtos.html

```
<!-- Seção de Produtos -->
<section>
  <h2>Produtos:</h2>

  <!-- Estado 1: Loading -->
  @if (carregando()) {
    <p>Carregando produtos...</p>
  }

  <!-- Estado 2: Erro -->
  @if (erro()) {
    <p style="color: red">
      {{ erro() }}
    </p>

    <button (click)="carregarProdutos()">Tentar novamente</button>
  }

  <!-- Estado 3: Sucesso -->
  @if (!carregando() && !erro()) {
    <p>Total de produtos: {{ totalProdutos() }}</p>

    <hr />

    <section class="grid-produtos">
      @for (item of produtos(); track $index) {
        <app-produto
          [nome]="item.nome"
          [preco]="item.preco"
          (produtoSelecionado)="exibirProduto($event)"
          (produtoAdicionado)="adicionarAoCarrinho($event)"
        />
      }
    </section>
  }
</section>
}
```

```
<h2>
  Produto selecionado:
  @if (produtoSelecionado()) {
    <strong style="color: green">{{ produtoSelecionado() }}</strong>
  } @else {
    <span style="color: gray">Nenhum</span>
  }
</h2>

<button mat-raised-button color="primary" (click)="adicionarProduto()">
  Adicionar Produto
</button>

<button mat-stroked-button color="accent" (click)="substituirProdutos()">
  Substituir Lista
</button>

<h2>Total de produtos: {{ totalProdutos() }}</h2>
<h3>Valor total: R$ {{ valorTotal().toFixed(2) }}</h3>
</section>

<!-- Seção do Carrinho -->
<section>
  <hr />

  <h2>Carrinho:</h2>

  @for (item of carrinhoFacade.itens(); track $index) {
    <p>{{ item.nome }} - R$ {{ item.preco.toFixed(2) }}</p>
  }

  <h3>Itens no carrinho: {{ carrinhoFacade.quantidade() }}</h3>
  <h3>Total do Carrinho: R$ {{ carrinhoFacade.total().toFixed(2) }}</h3>
</section>
```

Etapa 7 — Atualizar `checkout.ts`

Arquivo: `src/app/features/checkout/checkout/checkout.ts`

Substituir uso do `CarrinhoService` por `CarrinhoFacade` no componente de checkout.

itens

Lista de produtos via facade

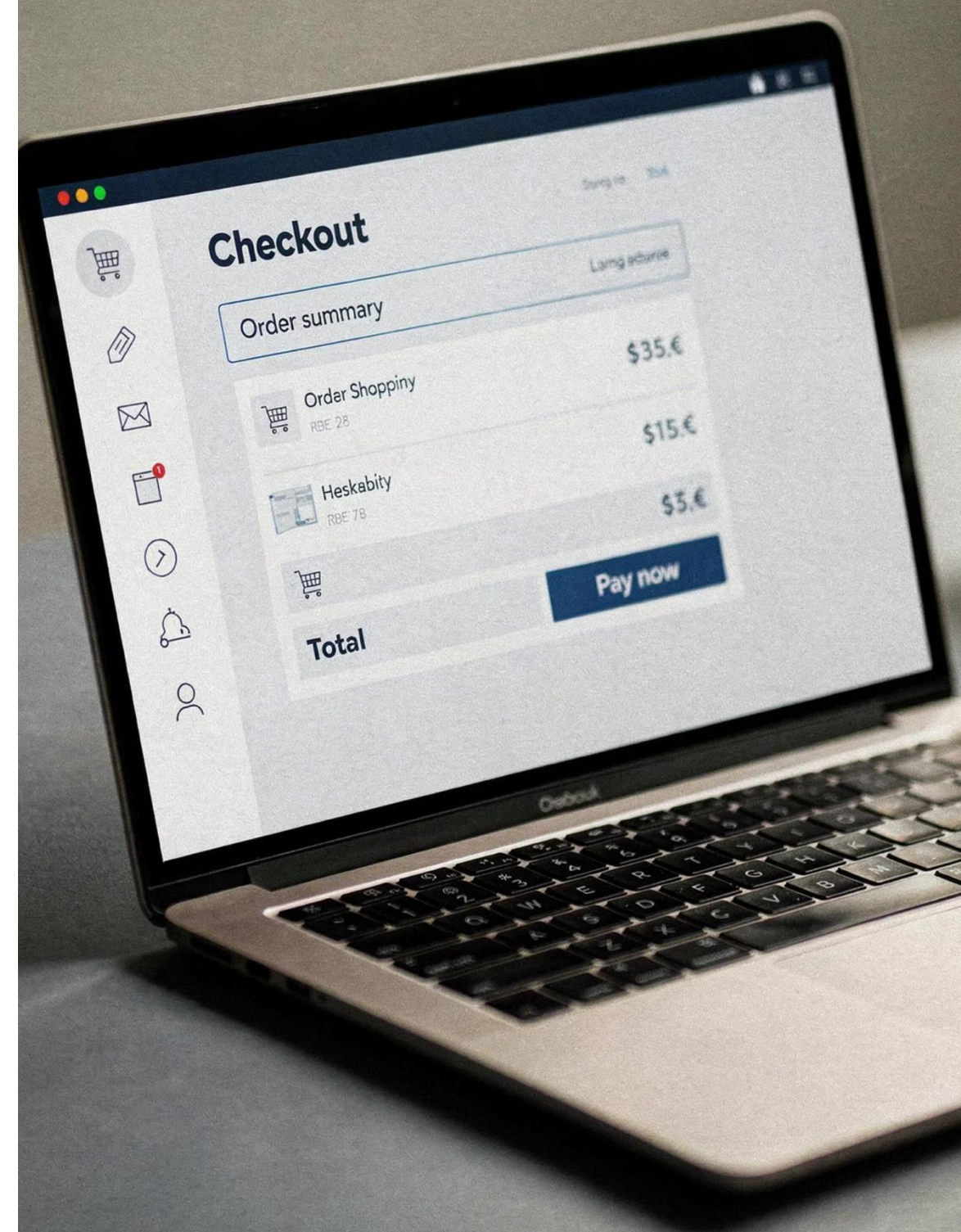
total

Valor total via facade

limpar carrinho

Ação delegada via facade

✔ Objetivo: manter o checkout desacoplado da implementação interna do service.



Código — checkout.ts

```
import { Component, inject, signal } from '@angular/core';
import {
  ReactiveFormsModule,
  FormGroup,
  FormControl,
  Validators,
  AbstractControl,
  ValidationErrors,
} from '@angular/forms';

import { CarrinhoFacade } from '../../../core/facades/carrinho.facade';

function nomeSemNumeros(control: AbstractControl): ValidationErrors | null {
  const valor = control.value;

  if (!valor) return null;

  if (/\\d/.test(valor)) {
    return { numeroInvalido: true };
  }
  return null;
}

@Component({
  selector: 'app-checkout',
  imports: [ReactiveFormsModule],
  templateUrl: './checkout.html',
  styleUrls: ['./checkout.css'],
})
export class Checkout {
  carrinhoFacade = inject(CarrinhoFacade);

  compraFinalizada = signal(false);

  formulario = new FormGroup({
    nome: new FormControl('', [Validators.required, Validators.minLength(3), nomeSemNumeros]),
    email: new FormControl('', [Validators.required, Validators.email]),
    endereco: new FormControl('', [Validators.required, Validators.minLength(5)]),
  });
```

src/app/features/checkout/checkout/checkout.ts

```
finalizar() {
  this.compraFinalizada.set(false);

  if (this.carrinhoFacade.carrinhoVazio()) {
    console.log('Não é possível finalizar uma compra com o carrinho vazio.');
    return;
  }

  if (this.formulario.invalid) {
    console.log('Formulário inválido');
    this.formulario.markAllAsTouched();
    return;
  }

  const dados = this.formulario.value;
  const itens = this.carrinhoFacade.itens();
  const total = this.carrinhoFacade.total();

  console.log('Compra finalizada com sucesso!');
  console.log('Dados do formulário:', dados);
  console.log('Itens do carrinho:', itens);
  console.log('Total da compra:', total);

  this.carrinhoFacade.limparCarrinho();
  this.formulario.reset();
  this.compraFinalizada.set(true);
}
```

Etapa 8 — Atualizar `checkout.html`

`src/app/features/checkout/checkout/checkout.html`

```
<h2>Checkout</h2>

@if (compraFinalizada()) {
  <p style="color: green">
    Compra finalizada com sucesso!
  </p>
}

@if (carrinhoFacade.carrinhoVazio()) {
  <p style="color: red">
    Seu carrinho está vazio. Adicione produtos antes de finalizar a compra.
  </p>
} @else {
  <section>
    <h3>Resumo do pedido</h3>

    @for (item of carrinhoFacade.itens(); track $index) {
      <p>{{ item.nome }} - R$ {{ item.preco.toFixed(2) }}</p>
    }

    <h3>Total: R$ {{ carrinhoFacade.total().toFixed(2) }}</h3>
  </section>
}

<form [formGroup]="formulario">
  <p>
    <input type="text" formControlName="nome" placeholder="Nome" />
  </p>
  @if (formulario.get('nome')?.touched && formulario.get('nome')?.invalid) {
    @if (formulario.get('nome')?.errors?.['required']) {
      <small>Nome é obrigatório</small>
    }
    @if (formulario.get('nome')?.errors?.['minlength']) {
      <small>Nome deve ter no mínimo 3 caracteres</small>
    }
    @if (formulario.get('nome')?.errors?.['numeroInvalido']) {
      <small>Nome não pode conter números</small>
    }
  }
}
```

```
<p>
  <input type="email" formControlName="email" placeholder="E-mail" />
</p>
@if (formulario.get('email')?.touched && formulario.get('email')?.invalid) {
  @if (formulario.get('email')?.errors?.['required']) {
    <small>Email é obrigatório</small>
  }

  @if (formulario.get('email')?.errors?.['email']) {
    <small>Email inválido</small>
  }
}
<p>
  <input type="text" formControlName="endereco" placeholder="Endereço" />
</p>
@if (formulario.get('endereco')?.touched && formulario.get('endereco')?.invalid) {
  @if (formulario.get('endereco')?.errors?.['required']) {
    <small>Endereço é obrigatório</small>
  }
  @if (formulario.get('endereco')?.errors?.['minlength']) {
    <small>Endereço deve ter no mínimo 5 caracteres</small>
  }
}
<p>
  <button type="button" (click)="finalizar()" [disabled]="formulario.invalid">
    Finalizar Compra
  </button>
</p>
</form>
```



Lapnmim

Wineo Secuerd

Seguin

Etapa 9 — Atualizar `login.ts`

Arquivo: `src/app/features/login/login/login.ts`

Substituir uso direto do `AuthService` por `AuthFacade` no componente de login.

1

Antes

```
this.authService.login(...)
```

2

Depois

```
this.authFacade.realizarLogin(...)
```



Objetivo: centralizar o acesso à autenticação através da facade, removendo dependência direta do `AuthService`.

Código — login.ts

src/app/features/login/login/login.ts

```
import { Component, inject, signal } from '@angular/core';
import { FormControl, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';
import { Router } from '@angular/router';

import { AuthFacade } from '../../../core/facades/auth.facade';

@Component({
  selector: 'app-login',
  imports: [ReactiveFormsModule],
  templateUrl: './login.html',
  styleUrls: ['./login.css'],
})
export class Login {
  private authFacade = inject(AuthFacade);
  private router = inject(Router);

  erroLogin = signal(false);

  formulario = new FormGroup({
    email: new FormControl('', [Validators.required, Validators.email]),
    senha: new FormControl('', [Validators.required, Validators.minLength(3)]),
  });
```

```
  entrar() {
    this.erroLogin.set(false);

    if (this.formulario.invalid) {
      this.formulario.markAllAsTouched();
      return;
    }

    const email = this.formulario.value.email ?? '';
    const senha = this.formulario.value.senha ?? '';

    const loginRealizado = this.authFacade.realizarLogin(email, senha);

    if (!loginRealizado) {
      this.erroLogin.set(true);
      return;
    }

    if (this.authFacade.ehAdmin()) {
      this.router.navigateByUrl('/admin');
      return;
    }

    this.router.navigateByUrl('/produtos');
  }
}
```

Etapa 10 — Atualizar `admin.ts`

Arquivo: `src/app/features/admin/admin/admin.ts`

Substituir uso de `AuthService` por `AuthFacade` no painel administrativo.

Usuário Atual

Leitura do usuário logado via facade

Verificação de Perfil

Checagem de admin via facade

Logout

Ação de logout delegada via facade

 O componente admin passa a depender apenas da `AuthFacade`, sem conhecer os detalhes internos do `AuthService`.

Código — admin.ts

src/app/features/admin/admin/admin.ts

```
import { Component, computed, inject, signal } from '@angular/core';
import { Router } from '@angular/router';
import { AuthFacade } from '../../../core/facades/auth.facade';

@Component({
  selector: 'app-admin',
  imports: [],
  templateUrl: './admin.html',
  styleUrls: ['./admin.css'],
})
export class Admin {

  // A área administrativa passa a ler dados do usuário autenticado.
  private authFacade = inject(AuthFacade);

  // Router usado para voltar para a área pública após logout.
  private router = inject(Router);

  // Signals simulando indicadores administrativos do sistema.
  totalProdutosCadastrados = signal(20);
  pedidosPendentes = signal(3);
  usuariosCadastrados = signal(8);

  // Computed signal derivado do usuário atual.
  usuarioAtual = this.authFacade.usuarioAtual;

  // Texto calculado para reforçar o perfil atualmente autenticado.
  mensagemPerfil = computed(() => {
    const usuario = this.usuarioAtual();
    if (!usuario) {
      return 'Nenhum usuário autenticado.';
    }
    return `Usuário autenticado como ${usuario.perfil}.`;
  });

  sair() {
    this.authFacade.sair();
    this.router.navigateByUrl('/login');
  }
}
```

Etapa 11 – Atualizar `auth.guard.ts`

Arquivo: `src/app/core/auth.guard.ts`

ANTES

Injeção direta de `AuthService`

DEPOIS

Injeção de `AuthFacade`

Regra mantida

→ Se o usuário **não estiver logado** → redireciona para a página de login

→ Se o usuário **estiver logado** → permite acesso à rota protegida

✅ O comportamento do guard permanece idêntico. Apenas a fonte de dados muda para a facade.

Código — auth.guard.ts

src/app/core/auth.guard.ts

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';

import { AuthFacade } from '../facades/auth.facade';

export const authGuard: CanActivateFn = () => {
  const authFacade = inject(AuthFacade);
  const router = inject(Router);

  if (authFacade.estaLogado()) {
    return true;
  }

  return router.createUrlTree(['/login']);
};
```

Etapa 12 – Atualizar `admin.guard.ts`

Arquivo: `src/app/core/admin.guard.ts`

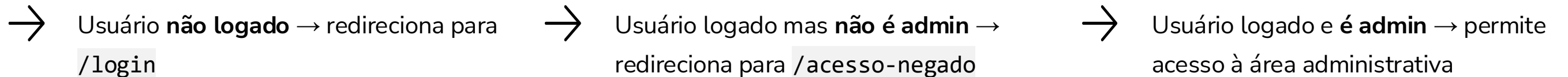
ANTES

Injeção direta de `AuthService`

DEPOIS

Injeção de `AuthFacade`

Fluxo de decisão mantido



Código — `admin.guard.ts`

`src/app/core/admin.guard.ts`

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';

import { AuthFacade } from '../facades/auth.facade';

export const adminGuard: CanActivateFn = () => {
  const authFacade = inject(AuthFacade);
  const router = inject(Router);

  if (!authFacade.estaLogado()) {
    return router.createUrlTree(['/login']);
  }

  if (!authFacade.ehAdmin()) {
    return router.createUrlTree(['/acesso-negado']);
  }

  // Se estiver logado e for admin, libera o acesso.
  return true;
};
```

Etapa 13 — Atualizar `http.interceptor.ts`

Arquivo: `src/app/core/interceptors/http.interceptor.ts`

ANTES

Injeção direta de `AuthService`

DEPOIS

Injeção de `AuthFacade`

O que é centralizado via facade

Token JWT

Acesso ao token para injeção nos headers HTTP

Logout

Ação de logout em caso de resposta 401



Objetivo: centralizar o acesso ao token e ao logout através da facade, desacoplando o interceptor do `AuthService`.

Código — http.interceptor.ts

src/app/core/interceptors/http.interceptor.ts

```
import { inject } from '@angular/core';
import { HttpInterceptorFn } from '@angular/common/http';
import { Router } from '@angular/router';
import { tap, catchError, throwError } from 'rxjs';

import { AuthFacade } from '../facades/auth.facade';

export const httpInterceptor: HttpInterceptorFn = (req, next) => {
  // Interceptor passa a obter dados de autenticação pela facade.
  const authFacade = inject(AuthFacade);

  // Router usado para redirecionamentos em erros de autenticação/autorização.
  const router = inject(Router);
  const token = authFacade.obterToken();

  console.log('REQUEST', req.url);

  const novaReq = token
    ? req.clone({
        headers: {
          Authorization: `Bearer ${token}`,
        },
      })
    : req;

  return next(novaReq).pipe(
    tap({
      next: (event) => console.log('RESPONSE:', event),
      error: (error) => console.error('ERRO:', error),
    }),
  );
}
```

```
catchError((error) => {
  console.error('ERRO GLOBAL:', error);

  // 401 -> ausência de autenticação ou token inválido.
  if (error.status === 401) {
    console.warn('Não autorizado. Faça login novamente.');
```

```
    authFacade.sair();
    router.navigateByUrl('/login');
  }

  // 403 -> usuario autenticado, mas sem permissão.
  if (error.status === 403) {
    console.warn('Acesso proibido. Perfil sem permissão.');
```

```
    router.navigateByUrl('/produtos');
  }

  // 500 -> erro interno do servidor
  if (error.status === 500) {
    console.warn('Erro interno do servidor!');
```

```
  }

  return throwError(() => error);
}));
};
```



Importante – Comportamento do Login

Nesta aula, o login continua sendo mantido apenas em memória (Signals).

Consequência

Ao atualizar a página ou acessar uma rota diretamente pela URL, o usuário perde o login.

Observação

Em aplicações Angular modernas com SSR/hydration, o estado em memória não é mantido entre requisições, o que reforça a necessidade de persistência (ex: localStorage ou cookies).

Resultado desta Aula

Após esta aula, o projeto passa a ter uma arquitetura muito mais profissional e preparada para crescer.



Arquitetura Profissional

Padrão utilizado em aplicações reais de mercado



Separação Clara

Responsabilidades bem definidas em cada camada



Componentes Desacoplados

Componentes, guards e interceptor passam a acessar regras globais por meio das facades.



Código Escalável

Estrutura preparada para crescer com o projeto

- ✔ Nenhuma funcionalidade foi alterada. A mudança foi totalmente estrutural. O projeto funciona exatamente igual, mas agora com uma arquitetura muito mais sólida.



Encerramento da Aula

Você evoluiu o projeto de um nível **funcional** para um nível **profissional**.

Este padrão de Facade é amplamente utilizado em aplicações reais de mercado. Ao separar o acesso à lógica de negócio dos componentes, você garante um código mais limpo, testável e fácil de manter.

O que foi feito

Criação das fachades de carrinho e autenticação, e atualização dos principais componentes, guards e interceptor envolvidos nesta arquitetura.

Dúvidas

?

